WALKING THROUGH AN IBFE MAIN FILE

IBAMR's `IBFE/explicit/ex7`, a collapsible channel, read from the first `#include` to `output_data`: what each block does, which input keys it reads, what the libraries behind it are, how the 2020 version differs, and the sponge layer that the input never turns on



`examples/IBFE/explicit/ex7/example.cpp`, 504 lines

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part IV • Chapter 23

Claude (Anthropic) was used to organize, combine, and format more than twelve years (2013–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

1. name the blocks every IBAMR main file has, in order, and say what each one does and which input-file block it reads;
2. read the C++ you meet there (namespaces, classes and their member functions, `Pointer<>`, headers, `static` globals, callbacks) well enough to follow any example, and find the rest in the documentation;
3. explain IBAMR's `ex7`: a channel with a flexible segment in its upper wall held by an external pressure, three cases, and what the run produces;
4. say what `IBTKInit`, `AppInitializer`, the solver objects, the gridding objects and the writers are, and what the time loop asks of them;
5. tell the 2020 version of the file from today's and update one to the other;
6. explain what the sponge layer does and why, in `ex7` as shipped, it does nothing.

> The 2020 lecture said: "The approach I will use can be applied to understanding other IBAMR examples." That is the point: one file read closely, then the same map on every other.

WHERE THIS CHAPTER COMES FROM

- "Lecture 8-IBFE-Going through a main file" (August 2020; 30 slides; `IBFE_Lecture_8`): the IBAMR stack, a C++ primer (namespaces, classes, headers), then `ex7` block by block in nine code screenshots with one- or two-line notes ("Pick your Navier Stokes solver", "Write initial visit and exodus files", "Not totally sure what sponge layer does"). Its C++ slides quoted `tutorialspoint.com` and its stack slides IBAMR's README: both are rewritten here in this series' words; the screenshots are transcribed from the 0.19.0 file and the 2020 differences listed.
- IBAMR 0.19.0, `examples/IBFE/explicit/ex7`: `example.cpp` (504 lines), three inputs `input2d.caseC`, `caseS2`, `caseSP`; `ibtk/AppInitializer.cpp`, `IBTKInit.cpp`, `SpongeLayerForceFunction.cpp` for what the calls do.
- The run: `caseC` built against 0.19.0 with the six-line `CMakeLists.txt` of chapter 15 and run to $t = 2$ s on four cores; its figures and cost are on slides 38–41.
- New here: the order-of-blocks map; the input-key aliases `AppInitializer` accepts; the three cases compared; the sponge layer explained from its source, and the key-name mismatch that leaves it off; the 2020-to-now table; the recipe for reading any other example.

THE STACK UNDER A MAIN FILE (1)

library	what it does in an IBFE program	where you meet it
SAMRAI	the Cartesian grid, its levels and patches, the input database, restart files, timers	`Pointer<Database>`, `PatchHierarchy`, `CartesianGridGeometry`, `GriddingAlgorithm`
PETSc	linear solvers and the MPI start-up; `-ksp_*` options	`petscsys.h`, `petsc_options_file`
libMesh	the finite element mesh, its systems and degrees of freedom, ExodusII output	`Mesh`, `MeshTools::Generation`, `EquationSystems`, `ExodusII_IO`
hypre	the multigrid preconditioners behind the pressure and velocity solves	the input's `stokes_solver_db` (chapter 16)
IBTK	IBAMR's toolkit: `AppInitializer`, `IBTKInit`, muParser functions, `IBTK_MPI`	`ibtk/*.h`
IBAMR	the IB methods and the fluid solvers	`IBFEMethod`, `IBExplicitHierarchyIntegrator`, `INSStaggeredHierarchyIntegrator`

The 2020 slide 2 listed these from IBAMR's README; the column on the right is what the list means when you read a main file.

THE STACK UNDER A MAIN FILE (2)

library	what it does	where you meet it
Boost	multi-dimensional arrays	`boost::multi_array` (chapter 22's post-processing)
Eigen	small dense linear algebra	inside IBAMR; `app_namespaces.h` brings it in
HDF5	binary files for restarts and state dumps	`HDFDatabase` (slide 36)
muParser	the expressions in the input (`"U0*tanh(2*t)"`)	`muParserCartGridFunction`, `muParserRobinBcCoefs`
Silo	the point method's Lagrangian output (Part III)	`lag_data.visit`
MPI	the processes; `mpirun -np 4`	`IBTK_MPI::getRank()`, `sumReduction`
VisIt	reads what the writers write	`dumps.visit`, `*.ex2`

- "IBAMR requires a number of third-party libraries. Sample build instructions are provided for a typical Linux installation and macOS. We offer experimental support the use of Windows 10 through the Windows Subsystem for Linux (WSL)" (2020, slide 3, from the README). autoibamr builds the whole stack (chapter 3); this series' WSL runs are the "experimental" route, and it works.
- A main file touches each library in one or two lines; the rest is inside IBAMR.

READING THE C++ (1): NAMESPACES AND CLASSES

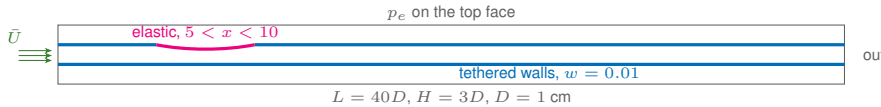
```
#include <ibamr/IBFEMethod.h>            // the class's declaration
#include <ibamr/app_namespaces.h>        // "using namespace" for IBAMR, IBTK, SAMRAI, libMesh, std, ...
Pointer<IBFEMethod> ib_method_ops =      // a reference-counted pointer to an object
    new IBFEMethod("IBFEMethod", input_db->getDatabase("IBFEMethod"), meshes, max_levels);
ib_method_ops->registerPK1StressFunction(data, 0);   // a member function, called through the pointer
IBFEMethod::PK1StressFcnData data(PK1_fcn);          // a type nested in the class; built from a function
static double kappa_s = 1.0e6;           // a file-wide variable the callbacks can see
namespace ModelData { ... }              // a named scope for the model's functions and constants
```

- A namespace is a named scope. `IBAMR::IBFEMethod` is the class `IBFEMethod` in the namespace `IBAMR`; two libraries can both have a `Point` without clashing. `app_namespaces.h` writes the `using namespace` lines for you, so the examples say `Pointer`, `Mesh`, `VectorValue` without prefixes. It must be included only in a main file, never in a header: a header that did it would force the choice on every file that includes it.
- A class bundles data with the functions that act on it. `ib_method_ops` is an object; `->` calls one of its member functions; `IBFEMethod::PK1StressFcnData` is a small class that lives inside `IBFEMethod`. The 2020 slides 5–6 explained both with a tutorial's words ("two persons with the same name, Zara"); for IBAMR the two examples above are the whole of it.

READING THE C++ (2): POINTERS, HEADERS, STATICS, CALLBACKS

- `Pointer<T>` is SAMRAI's smart pointer: it counts references and deletes the object when the last one goes; `new` creates the object; `setNull()` drops it. libMesh and the standard library use `std::unique_ptr` for the same job; the examples mix the two, one per library.
- A header file (`.h`) declares what a library offers; `#include` pastes it in. Every class you use needs its header; the compiler's "was not declared in this scope" means one is missing. "Including a header file is equal to copying the content of the header file" (2020, slide 9).
- `static` at file scope makes a variable private to this file; the model's constants are declared so, set from the input in `main`, and read by the callbacks, which cannot take extra arguments.
- A callback is a function you write and hand to IBAMR, which calls it when it needs a stress or a force (chapter 19): its signature is fixed by IBAMR, and `/*X*/` in the signature marks an argument the function does not use.
- Where to look: https://ibamr.github.io/docs, the search box, and the "Classes" tab: a class's page is its header made readable, with every member function and what it takes. "In general, I perform searches here to learn more about various functions and classes" (2020, slide 4), and "at some point, I recommend really learning C++."

WHAT EX7 IS: A COLLAPSIBLE CHANNEL



- Two thin walls, one element thick, built in the program with `build_square`: the lower at $y = D$, the upper at $y = 2D$, in a box $40D \times 3D$. The channel is the slot between them; the fluid above and below the walls is “outside”.
- The lower wall and the upper wall’s rigid parts are held by tethers; the segment $5 < x < 10$ of the upper wall is elastic (St. Venant–Kirchhoff, $E = 4.8 \times 10^6$ dyn/cm^2, $\nu = 0.45$) and untethered.
- A parabolic inflow of mean $\bar{U}$ enters at $x = 0$ between the walls, ramped up over $\tau = 1$ s; the outflow at $x = 40$ is traction-free; the top face carries the external pressure p_e as a normal stress, so the fluid above the upper wall presses on the elastic segment from outside while the channel’s own pressure presses from inside. Which way the segment goes depends on which is larger (slide 41).
- The lengths ($5D$ rigid, $5D$ flexible, $30D$ rigid) and the external pressure are the collapsible-channel problem of Luo and Pedley (1996), where a flexible segment buckles and oscillates as the flow and p_e compete; here the segment is a thin elastic wall rather than a membrane under tension.

EX7 (2): THE THREE CASES

	`caseC`	`caseS2`	`caseSP`
$\bar{U}$ (cm/s)	5	2	0.771
$\mathrm{Re} = \rho \bar{U} D/\mu$	500	200	77
p_e (dyn/cm^2)	48.75	8.60	17.55
E (dyn/cm^2)	4.79×10^6	1.76×10^5	7.18×10^5
inflow	velocity	velocity	pressure $p_u = 4.5$
`END_TIME`	$1000\,\mathrm{DT}$	100 s	100 s

- `caseC`'s `END_TIME` is 1000 steps, 0.02 s: a check that it runs. The two others run 100 s, five million steps; slide 40 runs `caseC` to 2 s.
- `caseSP` drives the flow by a pressure difference instead of a velocity: the inflow face's `acoef` is 0 and `bcoef` 1 inside the channel with $g = -p_u$ (a traction), and $\bar{U}$ is only used to size the time step.
- Water: $\rho = 1$, $\mu = 0.01$ in cgs; $\mathrm{DX} = 3/96 = 0.031$ cm on four levels at ratio 2; $\mathrm{DT} = 0.0125\,\mathrm{DX}/U_{\max} = 2 \times 10^{-5}$ s with $U_{\max} = 4\bar{U}$.
- $\mu_s = E/(2(1+\nu))$ and $\lambda_s = E\nu/((1+\nu)(1-2\nu))$ as in chapter 22; $\nu = 0.45$ makes the wall nearly incompressible, $\lambda_s = 9\mu_s$.

THE MAP (1): WHAT EVERY IBAMR MAIN FILE DOES, IN ORDER

	block	what it does	reads from the input
1	headers	the libraries' declarations; `app_namespaces.h` last	
2	`ModelData`	the model's constants and callbacks: stresses, forces, mappings	(set in `main`)
3	prototypes	`output_data`'s declaration, so `main` can call it	
4	start-up	`IBTKInit`: MPI, PETSc, SAMRAI, libMesh up and, at the end, down	`petsc_options_file`
5	`AppInitializer`	the command line, the input file, the log, restart, the writers' settings	`Main`
6	meshes	build or read the bodies; the vector of parts	`DX`, `MFAC`, `ELEM_TYPE`, `MESH_FILENAME`
7	solvers	the fluid solver, `IBFEMethod`, the IB time integrator	`Main.solver_type`, the three solver blocks
8	grid objects	geometry, hierarchy, tagging, box generation, load balancing, gridding	`CartesianGeometry`, `GriddingAlgorithm`, `StandardTagAndInitialize`, `LoadBalancer`
9	IBFE set-up	register the callbacks per part; `initializeFEEquationSystems()`	`IBFEMethod` (through the constructor)

THE MAP (2)

	block	what it does	reads from the input
10	ICs and BCs	muParser functions for the initial velocity and pressure; Robin coefficients per face	`VelocityInitialConditions`, `PressureInitialConditions`, `VelocityBcCoefs_d`
11	body force	an Eulerian force on the fluid (here the sponge layer)	`ForcingFunction` or `SpongeLayerForceFunction`
12	writers	VisIt for the grid, one ExodusII per part; append on restart	`viz_*`, `exodus_filename`
13	initialize	`initializeFEData()`, `initializePatchHierarchy()`; drop the initializer; log the input	
14	initial dump	frame 0	
15	time loop	step, then at intervals: viz, restart, timers, post-processing	the intervals
16	clean-up	delete the boundary objects; the braces end everything else	
17	`output_data`	whatever you measure (chapter 11, chapter 22)	`data_*`

- The course's `main.C` files (chapters 17–22) have the same seventeen blocks; IBAMR's IB examples (chapter 15) replace blocks 6 and 9 by the point-method's `IBStandardInitializer` and `IBMethod`.
- Blocks 4, 5, 7, 8, 12, 13 and 15 are copied unchanged from example to example; 2, 6, 9, 10, 11 and 17 are where a model lives.

BLOCK 1: THE HEADERS

```
// Config files
#include <SAMRAI_config.h>
// Headers for basic PETSc functions
#include <petscsys.h>
// Headers for basic SAMRAI objects
#include <BergerRigoutsos.h>
#include <CartesianGridGeometry.h>
#include <LoadBalancer.h>
#include <StandardTagAndInitialize.h>
// Headers for basic libMesh objects
#include <libmesh/boundary_info.h>
#include <libmesh/equation_systems.h>
#include <libmesh/exodusII_io.h>
#include <libmesh/mesh.h>
#include <libmesh/mesh_generation.h>
```

```
// Headers for application-specific algorithm/data structure
  objects
#include <ibamr/IBExplicitHierarchyIntegrator.h>
#include <ibamr/IBFEMethod.h>
#include <ibamr/INSCollocatedHierarchyIntegrator.h>
#include <ibamr/INSStaggeredHierarchyIntegrator.h>
#include <ibamr/SpongeLayerForceFunction.h>
#include <ibtk/AppInitializer.h>
#include <ibtk/IBTKInit.h>
#include <ibtk/IBTK_MPI.h>
#include <ibtk/libmesh_utilities.h>
#include <ibtk/muParserCartGridFunction.h>
#include <ibtk/muParserRobinBcCoefs.h>
// Set up application namespace declarations
#include <ibamr/app_namespaces.h>
```

- Five groups: SAMRAI's configuration and its four grid classes; PETSc's base header; libMesh's mesh, generator, systems and ExodusII writer; IBAMR's two fluid solvers, the IB integrator, `IBFEMethod` and the sponge layer; IBTK's start-up, initializer, MPI wrapper, libMesh helpers and the two muParser classes. Then the namespaces, last.
- The 2020 file began with `#include <IBAMR_config.h>` and `<IBTK_config.h>` and had no `IBTKInit.h` or `IBTK_MPI.h`: the headers are the quickest way to date a file (chapter 17).
- `mesh_generation.h` is here because `ex7` builds its walls in the program; a program that reads a mesh file needs `mesh.h` only. `SpongeLayerForceFunction.h` is the 2020 "example header file" (slide 10): "provides forcing at physical boundaries that weakly imposes homogeneous Dirichlet boundary conditions", slide 27.

BLOCK 2: MODELDATA, THE MODEL IN CALLBACKS

```
namespace ModelData
{
// Stress tensor function.
static double mu_s, lambda_s;
void upper_PK1_stress_function(TensorValue<double>& PP, const TensorValue<double>& FF, const libMesh::Point& /*X*/, const libMesh::
  Point& s,
                               Elem* const /*elem*/, const vector<const vector<double>*>& /*var_data*/,
                               const vector<const vector<VectorValue<double>>*>& /*grad_var_data*/, double /*time*/, void* /*ctx*/)
{
    if (s(0) > 5.0 && s(0) < 10.0)
    {
        static const TensorValue<double> II(1.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 1.0);
        const TensorValue<double> CC = FF.transpose() * FF;
        const TensorValue<double> EE = 0.5 * (CC - II);
        const TensorValue<double> SS = lambda_s * EE.tr() * II + 2.0 * mu_s * EE;
        PP = FF * SS;
    }
    else { PP.zero(); }
    return;
} // upper_PK1_stress_function
// Tether (penalty) force functions for lower and upper blocks.
static double kappa_s = 1.0e6;
void lower_tether_force_function(VectorValue<double>& F, ..., const libMesh::Point& X, const libMesh::Point& s, ...)
{   F = kappa_s * (s - X);  return; }
void upper_tether_force_function(VectorValue<double>& F, ..., const libMesh::Point& X, const libMesh::Point& s, ...)
{   if (s(0) > 5.0 && s(0) < 10.0) { F.zero(); } else { F = kappa_s * (s - X); }  return; }
} // namespace ModelData
using namespace ModelData;
```

BLOCK 2 (2): WHAT THE CALLBACKS SAY

- The St. Venant–Kirchhoff stress of chapter 19 (slide 13) on the elastic segment: $\mathbb{C} = \mathbb{F}^T\mathbb{F}$, $\mathbb{E} = \frac{1}{2}(\mathbb{C} - \mathbb{I})$, $\mathbb{S} = \lambda_s \operatorname{tr}\mathbb{E}\,\mathbb{I} + 2\mu_s\mathbb{E}$, $\mathbb{P} = \mathbb{F}\mathbb{S}$; and *zero* elsewhere. The rigid parts have no stiffness of their own, only the tether. `II` is written with nine entries, a 3D identity; in 2D libMesh's `TensorValue` ignores the third row and column.
- The upper tether is the complement: zero on the segment, $\kappa_s(\mathbf{s} - \mathbf{X})$ elsewhere. The lower wall is tethered everywhere and has no stress function at all: a part without stiffness (chapter 21), which is what a rigid wall held by a stiff tether should be.
- The segment is attached to the rigid parts only through the shared nodes at $x = 5$ and $x = 10$; the elastic stress there pulls on nodes the tether holds, so the segment is clamped.
- μ_s, λ_s and κ_s are `static` and set from the input in `main` (2020, slide 18: "kappa_s = target force stiffness"); the `1.0e6` is a default that the input overrides with $2.5\,\mathrm{DX}/\mathrm{DT}^2 = 2 \times 10^9$.
- `using namespace ModelData;` after the namespace lets `main` write `kappa_s` without the prefix; the course's files do the same.

BLOCK 4: THE START-UP, THEN AND NOW

2020 (slide 12, “Main function”):

```
int
main(int argc, char* argv[])
{
    // Initialize libMesh, PETSc, MPI, and SAMRAI.
    LibMeshInit init(argc, argv);
    SAMRAI_MPI::setCommunicator(PETSC_COMM_WORLD);
    SAMRAI_MPI::setCallAbortInSerialInsteadOfExit();
    SAMRAIManager::startup();
    { // cleanup dynamically allocated objects prior to shutdown
        ...
    }
    SAMRAIManager::shutdown();
    return 0;
}
```

0.19.0:

```
int
main(int argc, char* argv[])
{
    // Initialize IBAMR and libraries. Deinitialization is
  handled by this object as well.
    IBTKInit ibtk_init(argc, argv, MPI_COMM_WORLD);
    const LibMeshInit& init = ibtk_init.getLibMeshInit();
    { // cleanup dynamically allocated objects prior to
  shutdown
        ...
    } // cleanup dynamically allocated objects prior to
  shutdown
} // main
```

- `IBTKInit`’s constructor calls `PetscInitialize`, sets SAMRAI’s and IBTK’s communicators, raises SAMRAI’s patch-data limit to 2048, starts `SAMRAIManager` and libMesh; its destructor shuts them down in the right order. “Attempts to create a second IBTKInit object will result in a run time error.”
- `init` is what `Mesh` needs (`Mesh mesh(init.comm(), NDIM)`); the 2020 `Mesh mesh(NDIM)` is the old libMesh signature (chapter 17).
- The inner braces matter: every object made inside them is destroyed before the libraries shut down. Objects created outside them are destroyed after, and crash.

BLOCK 5: APPINITIALIZER (1): THE COMMAND LINE, THE LOG, THE WRITERS

```
// Parse command line options, set some standard options from the input
// file, initialize the restart database (if this is a restarted run),
// and enable file logging.
Pointer<AppInitializer> app_initializer = new AppInitializer(argc, argv, "IB.log");
Pointer<Database> input_db = app_initializer->getInputDatabase();
// Get various standard options set in the input file.
const bool dump_viz_data = app_initializer->dumpVizData();
const int viz_dump_interval = app_initializer->getVizDumpInterval();
const bool uses_visit = dump_viz_data && app_initializer->getVisItDataWriter();
#ifdef LIBMESH_HAVE_EXODUS_API
const bool uses_exodus = dump_viz_data && !app_initializer->getExodusIIFilename().empty();
#else
const bool uses_exodus = false;
if (!app_initializer->getExodusIIFilename().empty())
{
    plog << "WARNING: libMesh was compiled without Exodus support, so no "
         << "Exodus output will be written in this program.\n";
}
#endif
const string lower_exodus_filename = app_initializer->getExodusIIFilename("lower");
const string upper_exodus_filename = app_initializer->getExodusIIFilename("upper");
```

- `argv`: the input file, and for a restart the restart directory and number (`main2d input2d restart_IB2d 5000`); `"IB.log"` is the default log name, which `log_file_name` in `Main` overrides.
- `#ifdef LIBMESH_HAVE_EXODUS_API`: a libMesh built without ExodusII support compiles the program without the writer; autoibamr builds it with. `getExodusIIFilename("lower")`: chapter 21's prefix rule, `loweroutput.ex2`.

BLOCK 5 (2): RESTART, POST-PROCESSING AND TIMERS

```
const bool dump_restart_data = app_initializer->dumpRestartData();
const int restart_dump_interval = app_initializer->getRestartDumpInterval();
const string restart_dump_dirname = app_initializer->getRestartDumpDirectory();
const string restart_read_dirname = app_initializer->getRestartReadDirectory();
const int restart_restore_num = app_initializer->getRestartRestoreNumber();
const bool dump_postproc_data = app_initializer->dumpPostProcessingData();
const int postproc_data_dump_interval = app_initializer->getPostProcessingDataDumpInterval();
const string postproc_data_dump_dirname = app_initializer->getPostProcessingDataDumpDirectory();
if (dump_postproc_data && (postproc_data_dump_interval > 0) && !postproc_data_dump_dirname.empty())
{
    Utilities::recursiveMkdir(postproc_data_dump_dirname);
}
const bool dump_timer_data = app_initializer->dumpTimerData();
const int timer_dump_interval = app_initializer->getTimerDumpInterval();
```

- `getInputDatabase()` is the whole input file as a tree of databases; `getComponentDatabase("X")` is one block of it, handed to the object that reads it.
- “Determine intervals for writing restart files and post processing and writing data” (2020, slide 14). `dumpVizData()` is true when the viz interval is positive; the same pattern for restart, post-processing (“data”) and timers; `dumpRestartData` “return[s] a boolean value indicating whether to write restart data” (slide 15, from the documentation).
- `recursiveMkdir` makes the data folder here; the viz and restart folders are made by their writers. The log collects `plog` from every process if `log_all_nodes = TRUE`, else rank 0's.
- The restart read directory and number come from the command line, not the input, so the same input serves the fresh run and every restart of it.

BLOCK 5 (3): THE `MAIN` BLOCK'S KEYS AND THEIR ALIASES

"getRestartDumpInterval . . . can be defined in the input file as restart_interval, restart_dump_interval, or restart_write_interval" (2020, slide 15). From `AppInitializer.cpp`, the whole list:

setting	accepted keys (first found wins)	default
log file	`log_file_name`, `log_all_nodes`	the constructor's name; rank 0 only
viz interval	`viz_interval`, `viz_dump_interval`, `viz_write_interval`	0: no output
viz folder	`viz_dirname`, `viz_dump_dirname`, `viz_write_dirname`	
viz writers	`viz_writer`, `viz_writers`: `"VisIt"`, `"ExodusII"`, `"Silo"`, `"GMV"`	
VisIt files	`visit_number_procs_per_file`	1
ExodusII name	`exodus_filename`	`output.ex2`
restart interval, folder	`restart_interval` / `_dump_` / `_write_`; `restart_dirname` / `_dump_` / `_write_`	0
data (post-processing)	`data_interval` / `_dump_` / `_write_`; `data_dirname` / . . .	0
timers	`timer_interval` / `_dump_` / `_write_`	0
PETSc options	`petsc_options_file` (top level, not in `Main`)	

- The examples use the `_dump_` spelling; course inputs mix them, and all three work. A misspelt key is silently ignored, which is how an interval ends up at its default 0 and a run writes nothing.
- `viz_dump_interval = int(320.0/CFL_MAX)` in `ex7` is 3200 steps, 0.0625 s: the input language does arithmetic, and a key can be defined from an earlier one.

BLOCK 6: THE MESHES, BUILT IN THE PROGRAM

```
// Create a simple FE mesh.
const double dx = input_db->getDouble("DX");
const double ds = input_db->getDouble("MFAC") * dx;
const double D = 1.0;      // channel height (cm)
const double L = 40.0 * D; // channel length (cm)
const double w = 0.01 * D; // wall thickness (cm)
string elem_type = input_db->getString("ELEM_TYPE");
Mesh lower_mesh(init.comm(), NDIM);
MeshTools::Generation::build_square(lower_mesh, static_cast<int>(ceil(L / ds)), static_cast<int>(ceil(w / ds)),
                                    0.0, L, D - 0.5 * w, D + 0.5 * w, Utility::string_to_enum<ElemType>(elem_type));
lower_mesh.prepare_for_use();
Mesh upper_mesh(init.comm(), NDIM);
MeshTools::Generation::build_square(upper_mesh, static_cast<int>(ceil(L / ds)), static_cast<int>(ceil(w / ds)),
                                    0.0, L, 2.0 * D - 0.5 * w, 2.0 * D + 0.5 * w, Utility::string_to_enum<ElemType>(elem_type));
upper_mesh.prepare_for_use();
vector<MeshBase*> meshes(2);
meshes[0] = &lower_mesh;
meshes[1] = &upper_mesh;
mu_s = input_db->getDouble("MU_S");
lambda_s = input_db->getDouble("LAMBDA_S");
kappa_s = input_db->getDouble("KAPPA_S");
```

`build_square(mesh, nx, ny, xmin, xmax, ymin, ymax, type, gauss_lobatto_grid)` (the 2020 slide 17's table): a rectangle of `nx` by `ny` elements of `type`. "These immersed boundaries are geometrically relatively simple, so libmesh is used to make the meshes."

BLOCK 6 (2): WHAT THE NUMBERS COME TO

- `ds = MFAC * DX` is the element size: with `MFAC = 1` and $\mathrm{DX} = 0.031$, `nx = ceil(40/0.031) = 1280` and `ny = ceil(0.01/0.031) = 1`: each wall is 1280 `QUAD4` elements, one element thick, and 0.01 cm thin, a third of a grid cell. This is the case where `MFAC` matters (chapter 18): the program sizes the mesh from the grid, and a finer grid gives a finer wall.
- The walls are thinner than a cell, so the kernel smears each into a band about four cells wide; the channel the fluid sees is 1 cm high with soft edges. `IB_point_density = 3` puts three interaction points per cell along each wall element, enough that no fluid leaks between them (chapter 17).
- `prepare_for_use()` after building or reading; `init.comm()` is the communicator from the start-up.
- The vector of parts and their order (chapter 21): lower is part 0, upper part 1.
- The material constants are read here, after the `ModelData` statics exist and before any callback runs. `KAPPA_S = 2.5e1*DX/(DT^2)` in the input: 2×10^9, the stiffness scaled with the time step that chapter 19 (slide 26) discussed: the stiffest tether the explicit step allows.

BLOCK 7: THE SOLVERS

```
// Create major algorithm and data objects that comprise the
// application.  These objects are configured from the input database
// and, if this is a restarted run, from the restart database.
Pointer<INSHierarchyIntegrator> navier_stokes_integrator;
const string solver_type = app_initializer->getComponentDatabase("Main")->getString("solver_type");
if (solver_type == "STAGGERED")
{
    navier_stokes_integrator = new INSStaggeredHierarchyIntegrator(
        "INSStaggeredHierarchyIntegrator", app_initializer->getComponentDatabase("INSStaggeredHierarchyIntegrator"));
}
else if (solver_type == "COLLOCATED")
{
    navier_stokes_integrator = new INSCollocatedHierarchyIntegrator(
        "INSCollocatedHierarchyIntegrator", app_initializer->getComponentDatabase("INSCollocatedHierarchyIntegrator"));
}
else
{
    TBOX_ERROR("Unsupported solver type: " << solver_type << "\n" << "Valid options are: COLLOCATED, STAGGERED");
}
Pointer<IBFEMethod> ib_method_ops =
    new IBFEMethod("IBFEMethod", app_initializer->getComponentDatabase("IBFEMethod"), meshes,
                   app_initializer->getComponentDatabase("GriddingAlgorithm")->getInteger("max_levels"),
                   /*register_for_restart*/ true, restart_read_dirname, restart_restore_num);
Pointer<IBHierarchyIntegrator> time_integrator =
    new IBExplicitHierarchyIntegrator("IBHierarchyIntegrator", app_initializer->getComponentDatabase("IBHierarchyIntegrator"),
                                      ib_method_ops, navier_stokes_integrator);
```

BLOCK 7 (2): WHAT THE THREE OBJECTS ARE

- “Pick your Navier Stokes solver” (2020, slide 19): `Main.solver_type` chooses the staggered-grid solver (the one to use; chapter 6) or the collocated projection solver; each reads its own block, and both blocks are in every input so that the switch works. “Class INSStaggeredHierarchyIntegrator provides a staggered-grid solver for the incompressible Navier-Stokes equations on an AMR grid hierarchy” (slide 20, from the documentation).
- `INSHierarchyIntegrator` is the base class both share: the rest of the program talks to the base and does not care which it got. “Class IBFEMethod is an implementation of the abstract base class IBStrategy” (slide 21) in the same way: `IBMethod` (points), `IBFEMethod` (elements) and `ConstraintIBMethod` (chapter 14) all fit the same `IBExplicitHierarchyIntegrator`.
- `IBFEMethod`’s arguments: its block, the parts, `max_levels` from the gridding block (so that the two agree), and the restart settings. `register_for_restart = true` saves its finite element data with the restart files.
- `IBExplicitHierarchyIntegrator` owns the time step: it asks the fluid solver and the IB object in turn (chapter 5’s loop). Its block has `start_time`, `end_time`, `dt_max`, `num_cycles`, `regrid_cfl_interval`, `error_on_dt_change`.
- `TBOX_ERROR` prints and aborts on every process: the way IBAMR reports a bad input.

BLOCK 8: THE GRID OBJECTS

```
Pointer<CartesianGridGeometry<NDIM>> grid_geometry = new CartesianGridGeometry<NDIM>(
    "CartesianGeometry", app_initializer->getComponentDatabase("CartesianGeometry"));
Pointer<PatchHierarchy<NDIM>> patch_hierarchy = new PatchHierarchy<NDIM>("PatchHierarchy", grid_geometry);
Pointer<StandardTagAndInitialize<NDIM>> error_detector =
    new StandardTagAndInitialize<NDIM>("StandardTagAndInitialize", time_integrator,
                                       app_initializer->getComponentDatabase("StandardTagAndInitialize"));
Pointer<BergerRigoutsos<NDIM>> box_generator = new BergerRigoutsos<NDIM>();
Pointer<LoadBalancer<NDIM>> load_balancer =
    new LoadBalancer<NDIM>("LoadBalancer", app_initializer->getComponentDatabase("LoadBalancer"));
Pointer<GriddingAlgorithm<NDIM>> gridding_algorithm =
    new GriddingAlgorithm<NDIM>("GriddingAlgorithm", app_initializer->getComponentDatabase("GriddingAlgorithm"),
                                error_detector, box_generator, load_balancer);
```

Six SAMRAI objects, always these six, in this order:

- `CartesianGridGeometry`: the box, its level-0 cells, which directions are periodic (`domain_boxes`, `x_lo`, `x_up`, `periodic_dimension`; chapter 7). `PatchHierarchy`: the levels and their patches; empty until `initializePatchHierarchy`.
- `StandardTagAndInitialize`: asks the time integrator which cells to refine (`tagging_method = "GRADIENT_DETECTOR"`: the integrator's own rule, vorticity thresholds and Lagrangian points). `BergerRigoutsos`: turns tagged cells into rectangular patches. `LoadBalancer`: hands the patches to processes (`bin_pack_method = "SPATIAL"`).
- `GriddingAlgorithm`: runs the three above: `max_levels`, `ratio_to_coarser`, patch sizes, `efficiency_tolerance`, `combine_efficiency` (chapter 22, slide 20). `NDIM` is set when the program is compiled for 2D or 3D (`IBAMR::IBAMR2d` against `IBAMR3d`); the same source builds both.

BLOCK 9: CONFIGURE THE IBFE SOLVER

```
// Configure the IBFE solver.
IBFEMethod::LagBodyForceFcnData lower_tether_force_data(lower_tether_force_function);
ib_method_ops->registerLagBodyForceFunction(lower_tether_force_data, 0);
IBFEMethod::LagBodyForceFcnData upper_tether_force_data(upper_tether_force_function);
IBFEMethod::PK1StressFcnData upper_PK1_stress_data(upper_PK1_stress_function);
ib_method_ops->registerLagBodyForceFunction(upper_tether_force_data, 1);
ib_method_ops->registerPK1StressFunction(upper_PK1_stress_data, 1);
ib_method_ops->initializeFEEquationSystems();
EquationSystems* lower_equation_systems = ib_method_ops->getFEDataManager(0)->getEquationSystems();
EquationSystems* upper_equation_systems = ib_method_ops->getFEDataManager(1)->getEquationSystems();
```

“We reviewed this part when looking at multiple boundaries” (2020, slide 22): chapter 21.

- Each callback is wrapped in its `FcnData` (chapter 19: the quadrature order and the systems it reads live there), then registered on a part. Part 0 gets a tether only; part 1 a tether and a stress.
- `initializeFEEquationSystems()` creates, for each part, the libMesh systems that hold the positions, velocities and forces of the nodes; after it, no more registrations (chapter 22’s mapping rule).
- `getFEDataManager(part)->getEquationSystems()` is the handle the writers and the post-processing use; one per part.
- What is *not* here: a coordinate mapping (the walls are built where they stand), an initial velocity, a surface force: `ex7` needs none. The full list of what can be registered is chapter 19, slide 20.

BLOCK 10: INITIAL CONDITIONS AND BOUNDARY CONDITIONS

```
// Create Eulerian initial condition specification objects.
if (input_db->keyExists("VelocityInitialConditions"))
{
    Pointer<CartGridFunction> u_init = new muParserCartGridFunction(
        "u_init", app_initializer->getComponentDatabase("VelocityInitialConditions"), grid_geometry);
    navier_stokes_integrator->registerVelocityInitialConditions(u_init);
}
if (input_db->keyExists("PressureInitialConditions"))
{ ... registerPressureInitialConditions(p_init); }
// Create Eulerian boundary condition specification objects.
vector<RobinBcCoefStrategy<NDIM>*> u_bc_coefs(NDIM, nullptr);
const bool periodic_domain = grid_geometry->getPeriodicShift().min() > 0;
if (!periodic_domain)
{
    for (unsigned int d = 0; d < NDIM; ++d)
    {
        const std::string bc_coefs_name = "u_bc_coefs_" + std::to_string(d);
        const std::string bc_coefs_db_name = "VelocityBcCoefs_" + std::to_string(d);
        u_bc_coefs[d] = new muParserRobinBcCoefs(bc_coefs_name, app_initializer->getComponentDatabase(bc_coefs_db_name),
  grid_geometry);
    }
    navier_stokes_integrator->registerPhysicalBoundaryConditions(u_bc_coefs);
}
```

BLOCK 10 (2): WHAT EX7 PRESCRIBES

- A `muParserCartGridFunction` turns an input block of expressions (`function_0 = "..."`) into a field on the grid; registered as the initial velocity or pressure. `ex7`'s inputs have neither block, so the fluid starts at rest and the `if` skips. "Read in velocity or pressure boundary conditions from input file" (2020, slide 23) should read *initial* conditions.
- `muParserRobinBcCoefs`: one object per velocity component, reading `VelocityBcCoefs_d`, the `acoef`, `bcoef`, `gcoef` strings for each face (chapter 7). A periodic box needs none, and `getPeriodicShift().min() > 0` is the test for "periodic in every direction".
- `ex7`'s faces (`caseC`): at $x = 0$ the parabola $1.5\bar{U}\,(y - y_l)(y - y_u)/\ldots$ between the walls, ramped by $\frac{1}{2}(1 - \cos \pi t/\tau)$ for $t < \tau = 1$, and no slip outside them; at $x = 40$ no slip outside the channel and a zero traction inside (free outflow); at $y = 0$ and $y = 3$ no slip for u and a normal traction for v: 0 below, $-p_e$ (ramped) above. `caseSP` replaces the inflow velocity by a traction $-p_u$.
- The expressions use `X1` (the y coordinate), `t`, and the block's own constants (`X1_lower = D + 0.5*w + DX`: the first cell inside the wall); chapter 22's rule that a block sees only its own constants.
- The objects are raw pointers, deleted at the end of `main`; the 2020 version wrote `static_cast<RobinBcCoefStrategy<NDIM>*>(NULL)` where today's writes `nullptr`.

BLOCK 11: THE BODY FORCE, AND THE SPONGE LAYER

```
// Create Eulerian body force function specification objects.
time_integrator->registerBodyForceFunction(
    new SpongeLayerForceFunction("SpongeLayerForceFunction", app_initializer->getComponentDatabase("SpongeLayerForceFunction"),
                                 navier_stokes_integrator, grid_geometry));
```

From `SpongeLayerForceFunction.cpp` (0.19.0):

```
const std::string forcing_enabled_key = "forcing_enabled_" + std::to_string(location_index);   // 0..2*NDIM-1
if (input_db->keyExists(forcing_enabled_key)) d_forcing_enabled[location_index] = input_db->getBoolArray(forcing_enabled_key);
const std::string width_key = "width_" + std::to_string(location_index);
...
const double kappa = cycle_num >= 0 ? 0.5 * rho / dt : 0.0;
...
(*F_data)(i, d) = smooth_kernel((x - x_bdry) / d_width[location_index]) * kappa * (0.0 - U);
// smooth_kernel(r) = |r| < 1 ? 0.5 * (cos(pi r) + 1) : 0
```

and `ex7`'s input:

```
SpongeLayerForceFunction {
   enable_forcing_0 = 0 , 1
   enable_forcing_1 = 0 , 1
   width_0 = DX0
   width_1 = DX0
}
```

BLOCK 11 (2): WHAT THE SPONGE LAYER DOES, AND WHY HERE IT DOES NOTHING

- What it does. An Eulerian body force, registered like any other (`ForcingFunction` with muParser in the course inputs; chapter 7), that within a distance `width` of a chosen face damps a chosen velocity component toward zero: $f = \frac{\rho}{2\Delta t}\, K(x/w)\,(0-u)$, with K a raised cosine. "Weakly imposes homogeneous Dirichlet boundary conditions": it kills the velocity component without the solver's boundary machinery. Used to stop recirculation from re-entering at an outflow, or to damp the tangential velocity at an inflow.
- The keys: `forcing_enabled_<face>` is a list of `NDIM` booleans, one per velocity component; `width_<face>` a length; faces 0–3 are $x_{\mathrm{lo}}, x_{\mathrm{hi}}, y_{\mathrm{lo}}, y_{\mathrm{hi}}$.
- "Not totally sure what sponge layer does" (2020, slide 25): in `ex7`, nothing. The input spells the keys `enable_forcing_*`; the class reads `forcing_enabled_*`, finds nothing, and leaves every face off. The run is the same with the block deleted. The intent was: damp v in one coarse cell at the inflow and the outflow. To get it, rename the keys.
- A misspelt key that is silently ignored is the commonest kind of input bug (slide 18): the input's `printClassData` in the log marks every key it read with `// input used`; an unmarked key was never read.

BLOCK 12: THE WRITERS

```
// Set up visualization plot file writers.
Pointer<VisItDataWriter<NDIM>> visit_data_writer = app_initializer->getVisItDataWriter();
if (uses_visit) { time_integrator->registerVisItDataWriter(visit_data_writer); }
std::unique_ptr<ExodusII_IO> lower_exodus_io = uses_exodus ? std::make_unique<ExodusII_IO>(lower_mesh) : nullptr;
std::unique_ptr<ExodusII_IO> upper_exodus_io = uses_exodus ? std::make_unique<ExodusII_IO>(upper_mesh) : nullptr;
// Check to see if this is a restarted run to append current exodus files
if (uses_exodus)
{
    const bool from_restart = RestartManager::getManager()->isFromRestart();
    lower_exodus_io->append(from_restart);
    upper_exodus_io->append(from_restart);
}
```

- The VisIt writer comes from the initializer (it read `viz_dump_dirname`); the integrator registers its fields with it (`output_U`, `output_P`, ... in the solver block decide which). One ExodusII writer per part, on that part's mesh; `append` on restart (chapter 21: rank 0 only, which `ex7` does not do).
- "Visualization file writers" was the 2020 slide 26, a screenshot with that title and nothing else; the `unique_ptr` line was then `std::unique_ptr<ExodusII_IO> lower_exodus_io(uses_exodus ? new ExodusII_IO(lower_mesh) : NULL)`.
- On a restart this code has every process reopen both files, and with more than one process the restarted files cannot be opened; the series' examples let only processor 0 reopen them (chapter 21).

BLOCKS 13–14: INITIALIZATION AND THE FIRST FRAME

```
// Initialize hierarchy configuration and data on all patches.
ib_method_ops->initializeFEData();
time_integrator->initializePatchHierarchy(patch_hierarchy, gridding_algorithm);
// Deallocate initialization objects.
app_initializer.setNull();
// Print the input database contents to the log file.
plog << "Input database:\n";
input_db->printClassData(plog);
// Write out initial visualization data.
int iteration_num = time_integrator->getIntegratorStep();
double loop_time = time_integrator->getIntegratorTime();
if (dump_viz_data)
{
    pout << "\n\nWriting visualization files...\n\n";
    if (uses_visit) { time_integrator->setupPlotData(); visit_data_writer->writePlotData(patch_hierarchy, iteration_num, loop_time);
   }
    if (uses_exodus)
    {
        lower_exodus_io->write_timestep(lower_exodus_filename, *lower_equation_systems, iteration_num / viz_dump_interval + 1,
  loop_time);
        upper_exodus_io->write_timestep(upper_exodus_filename, *upper_equation_systems, iteration_num / viz_dump_interval + 1,
  loop_time);
    }
}
```

BLOCKS 13–14 (2): WHAT HAPPENS BEFORE THE FIRST STEP

- `initializeFEData()` fills the systems with the nodes' positions; `initializePatchHierarchy()` builds the levels for the first time, tagging round the bodies, and sets the initial fields. Order: FE data first, so that the Lagrangian points exist when the grid is tagged.
- `app_initializer.setNull()`: the initializer is done; dropping it frees what it held. `printClassData` writes the input back into the log with `// input used` marks.
- `getIntegratorStep()` and `getIntegratorTime()` are 0 for a fresh run and the restart's values for a restarted one; the first dump is frame `iteration_num / viz_dump_interval + 1`: ExodusII time states count from 1.
- "Write initial visit and exodus files" (2020, slide 27): frame 0 is the undeformed walls and the fluid at rest, the reference every later frame is compared with.

BLOCK 15: THE TIME LOOP

```
// Main time step loop.
double loop_time_end = time_integrator->getEndTime();
double dt = 0.0;
while (!IBTK::rel_equal_eps(loop_time, loop_time_end) && time_integrator->stepsRemaining())
{
    iteration_num = time_integrator->getIntegratorStep();
    loop_time = time_integrator->getIntegratorTime();
    pout << "\n";
    pout << "++++++++++++++++++++++++++++++++++++++++++++++++++\n";
    pout << "At beginning of timestep # " << iteration_num << "\n";
    pout << "Simulation time is " << loop_time << "\n";
    dt = time_integrator->getMaximumTimeStepSize();
    time_integrator->advanceHierarchy(dt);
    loop_time += dt;
    pout << "\n";
    pout << "At end       of timestep # " << iteration_num << "\n";
    pout << "Simulation time is " << loop_time << "\n";
    pout << "++++++++++++++++++++++++++++++++++++++++++++++++++\n";
    pout << "\n";
    // At specified intervals, write visualization and restart files,
    // print out timer data, and store hierarchy data for post
    // processing.
    iteration_num += 1;
    const bool last_step = !time_integrator->stepsRemaining();
    ...   // the output block, next slide
}
```

BLOCK 15 (2): WHAT THE LOOP DOES

- Two lines do the work: `getMaximumTimeStepSize()` (the smaller of `dt_max`, the CFL limit, and `grow_dt` times the last step; with `error_on_dt_change = TRUE` a change after the first steps aborts) and `advanceHierarchy(dt)`, one IB time step: spread the force, solve the fluid, interpolate the velocity, move the structure (chapter 5), with regridding when `regrid_cfl_interval` says so.
- The loop ends when the time reaches `end_time` within rounding (`rel_equal_eps`) or the integrator has no steps left (`max_integrator_steps`, if set). The 2020 line used `MathUtilities<double>::equalEps`: the same test, the old name.
- `pout` prints on rank `0` to the screen and the log; `plog` to the log only. The `+++` banner is what `grep "Simulation time"` finds when you check a job (chapter 16). “Write out information such as time step number, simulation time, etc.” (2020, slide 28).
- Everything the course added to a main file (the force integrals of chapter 11, `update_target_point_positions` of chapter 9) goes either into the callbacks or into this loop’s output block.

BLOCK 15 (3): THE OUTPUT BLOCK, AND THE END OF MAIN

```
    if (dump_viz_data && (iteration_num % viz_dump_interval == 0 || last_step))
    {
        pout << "\nWriting visualization files...\n\n";
        if (uses_visit) { time_integrator->setupPlotData(); visit_data_writer->writePlotData(patch_hierarchy, iteration_num,
  loop_time); }
        if (uses_exodus)
        {
            lower_exodus_io->write_timestep(lower_exodus_filename, *lower_equation_systems, iteration_num / viz_dump_interval + 1,
  loop_time);
            upper_exodus_io->write_timestep(upper_exodus_filename, *upper_equation_systems, iteration_num / viz_dump_interval + 1,
  loop_time);
        }
    }
    if (dump_restart_data && (iteration_num % restart_dump_interval == 0 || last_step))
    {
        pout << "\nWriting restart files...\n\n";
        RestartManager::getManager()->writeRestartFile(restart_dump_dirname, iteration_num);
        ib_method_ops->writeFEDataToRestartFile(restart_dump_dirname, iteration_num);
    }
    if (dump_timer_data && (iteration_num % timer_dump_interval == 0 || last_step))
    {
        pout << "\nWriting timer data...\n\n";
        TimerManager::getManager()->print(plog);
    }
    if (dump_postproc_data && (iteration_num % postproc_data_dump_interval == 0 || last_step))
    {
        pout << "\nWriting state data...\n\n";
        output_data(patch_hierarchy, navier_stokes_integrator, upper_mesh, upper_equation_systems,
                    iteration_num, loop_time, postproc_data_dump_dirname);
    }
}   // end of the time loop
```

BLOCK 15 (4): THE FOUR IFS, AND THE END OF MAIN

```
// Cleanup Eulerian boundary condition specification objects (when necessary).
for (unsigned int d = 0; d < NDIM; ++d) delete u_bc_coefs[d];
} // cleanup dynamically allocated objects prior to shutdown
} // main
```

- Four `if`s with the same shape: at the interval, or at the last step. The viz frame number is `iteration_num / viz_dump_interval + 1`; the restart files are named by the step, which is the number you give on the command line to continue.
- `writeRestartFile` saves the grid and the fluid; `writeFEDataToRestartFile` the finite element data of every part (chapter 21: both are needed). The timers (`TimerManager`) print where the time goes, with the `TimerManager` block choosing which.
- `output_data` here gets the *upper* mesh only: a two-part program with a one-part post-processing function, the 2015 note of chapter 21. Nothing is lost in `ex7`, because `data_dump_interval = 0`.
- The last lines: delete the boundary objects (made with `new`, not `Pointer`), close the braces, and let `IBTKInit`'s destructor shut the libraries down. There is no `return 0`: C++ returns 0 from `main` by default.
- The 2020 slide 29 showed this block without a title; its caption on slide 28 covers both: “Write out information such as time step number, simulation time, etc.”

BLOCK 17: EX7'S OUTPUT_DATA, A STATE DUMP

```
void output_data(Pointer<PatchHierarchy<NDIM>> patch_hierarchy, Pointer<INSHierarchyIntegrator> navier_stokes_integrator,
                 Mesh& mesh, EquationSystems* equation_systems, const int iteration_num, const double loop_time,
                 const string& data_dump_dirname)
{
    plog << "writing hierarchy data at iteration " << iteration_num << " to disk" << endl;
    // Write Cartesian data.
    string file_name = data_dump_dirname + "/" + "hier_data.";
    char temp_buf[128];
    std::snprintf(temp_buf, sizeof(temp_buf), "%05d.samrai.%05d", iteration_num, IBTK_MPI::getRank());  file_name += temp_buf;
    Pointer<HDFDatabase> hier_db = new HDFDatabase("hier_db");  hier_db->create(file_name);
    VariableDatabase<NDIM>* var_db = VariableDatabase<NDIM>::getDatabase();
    ComponentSelector hier_data;
    hier_data.setFlag(var_db->mapVariableAndContextToIndex(navier_stokes_integrator->getVelocityVariable(), navier_stokes_integrator
  ->getCurrentContext()));
    hier_data.setFlag(var_db->mapVariableAndContextToIndex(navier_stokes_integrator->getPressureVariable(), navier_stokes_integrator
  ->getCurrentContext()));
    patch_hierarchy->putToDatabase(hier_db->putDatabase("PatchHierarchy"), hier_data);
    hier_db->putDouble("loop_time", loop_time);
    hier_db->putInteger("iteration_num", iteration_num);
    hier_db->close();
    // Write Lagrangian data.
    file_name = data_dump_dirname + "/" + "fe_mesh.";
    std::snprintf(temp_buf, sizeof(temp_buf), "%05d", iteration_num);  file_name += temp_buf;  file_name += ".xda";
    mesh.write(file_name);
    file_name = data_dump_dirname + "/" + "fe_equation_systems.";
    std::snprintf(temp_buf, sizeof(temp_buf), "%05d", iteration_num);  file_name += temp_buf;
    equation_systems->write(file_name, (EquationSystems::WRITE_DATA | EquationSystems::WRITE_ADDITIONAL_DATA));
    return;
} // output_data
```

BLOCK 17 (2): MEASUREMENT OR STATE DUMP

- “This function varies a lot, but is used to output .curve files for velocities, forces, and other data. I recommend looking at a few examples” (2020, slide 30). Two kinds exist: a *measurement* (the course’s force integrals, chapter 11 and chapter 22) and, as here, a *state dump*: the velocity and pressure of the whole hierarchy to an HDF5 file per process (`hier_data.00320.samrai.00000`) and the mesh and its systems in libMesh’s own formats (`fe_mesh.00320.xda`, `fe_equation_systems.00320`).
- A state dump is what you read back into your own program or into Python for analysis that VisIt cannot do (chapter 33); the IB examples’ `hier_data` folders are the same thing.
- `ComponentSelector` picks which variables go to the file: the velocity and the pressure in their *current* context (the values at the end of the step; `new` and `scratch` are the integrator’s working copies).
- To measure a force instead, replace the body with chapter 22’s loop over elements; the signature stays, so nothing else in the file changes. The course’s `postprocess_data` of chapters 17 and 21 is this function under another name.

RUNNING IT: THE BUILD AND THE THREE CASES

```
# a copy of ex7 built on its own (chapter 15's six-line CMakeLists.txt)
cp -r $IBAMR_SRC/examples/IBFE/explicit/ex7 ibfe_ex7 && cd ibfe_ex7
cat > CMakeLists.txt <<EOF
cmake_minimum_required(VERSION 3.15...3.30)
project(ibfe_ex7 LANGUAGES C CXX)
find_package(IBAMR 0.16.0 REQUIRED)
add_executable(main2d example.cpp)
target_link_libraries(main2d PRIVATE IBAMR::IBAMR2d)
EOF
source ~/autoibamr/configuration/enable.sh
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release && cmake --build build -j 4
cp input2d.caseC input2d        # then END_TIME = 2.0
mpirun -np 4 ./build/main2d input2d
```

- The example has no `Makefile` of its own: inside IBAMR's tree it is built by `IBAMR_ADD_EXAMPLE`; on its own, the six lines above (the same file as IBFE/08 and IBFE/10).
- The input is chosen by copying one case to `input2d`, or by giving its name on the command line.
- `END_TIME = 1000*DT` in `caseC` is IBAMR's own smoke test, the first thing to run after a build. The ramp $\tau = 1$ s means nothing moves in those 0.02 s; the figures on the next slides are from a run to 2 s.

RUNNING IT (2): THE COST

grid	160×12 on level 0, four levels at ratio 2, $\mathrm{DX} = 1/32$; the finest level follows the walls
walls	2×1280 `QUAD4`, one element thick; `IB_point_density = 3`
step	$\mathrm{DT} = 1.95 \times 10^{-5}$ s; 51200 steps per second of channel
speed	28 steps per second on four cores (desktop PC, WSL): 1 s of channel in 30 minutes; 2 s in 60
output	a frame every 3200 steps (0.0625 s): `visit_dump.NNNNN`, `loweroutput.ex2`, `upperoutput.ex2`; 33 frames, 81 MB, plus 29 MB of restart files
regrids	831 in 102400 steps: a wall that moves regrids the hierarchy every 120 steps or so (`regrid_cfl_interval = 0.5`)
`caseS2`, `caseSP` to 100 s	five million steps: a cluster job of days, with restarts every `restart_dump_interval` steps

- The time step is set by the tether: $\kappa_s = 25\,\mathrm{DX}/\mathrm{DT}^2$ is as stiff as the explicit step allows, and DT itself is $0.0125\,\mathrm{DX}/U_{\max}$, a CFL of 0.0125. A softer wall could take a larger step; chapter 19, slide 26.
- A long channel at this resolution is a 2D problem that costs like a small 3D one (chapter 22): $123\,000$ fine cells if the finest level covered everything; the vorticity tagging keeps it to the walls and the wake, and the frequent regrids are part of the price of a moving wall.

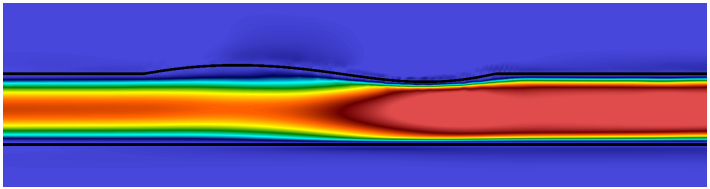
WHAT YOU SHOULD SEE (1)



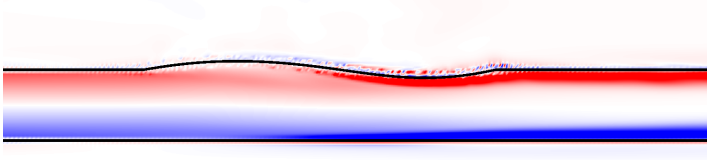
The whole channel at $t = 2$ s (`caseC`, $\mathrm{Re} = 500$): u from 0 to 10 cm/s; the two walls in black. The parabola enters at the left; the elastic segment ($5 < x < 10$) is swinging back from an outward bulge, and the fluid it releases runs down the channel faster than the inflow (the red slug).

- The box is 40 long and 3 high; the picture is to scale. The fluid above and below the walls is at rest except where the walls move; the channel between them carries the stream. Four grid levels follow the walls, as chapter 22's levels picture showed for the plate.
- What happened (from `upperoutput.ex2`): as the ramp raised the inflow, the pressure upstream of the segment rose to 87 dyn/cm^2, above $p_e = 49$, and by $t = 1$ the segment had bulged *outward* by 0.59 cm; by $t = 1.5$ it was at 0.51, and at $t = 2$ it is an S-shape, 0.12 out upstream and 0.11 in downstream. Luo and Pedley's steady collapse needs the channel pressure below p_e: a longer run, or `caseS2`'s slower stream and softer wall.

WHAT YOU SHOULD SEE (2): THE ELASTIC SEGMENT

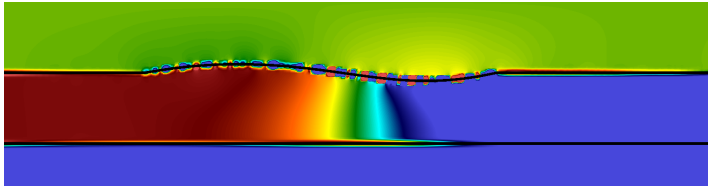


u (0 to 10 cm/s) near the segment, $3 < x < 13$, at $t = 2$ s: the wall is out upstream and in downstream, and the stream speeds up where the channel narrows.



Vorticity (± 50 s^{-1}) in the same window: the boundary layers on both walls (opposite signs), and the speckle of a thin wall that moves through the grid.

WHAT YOU SHOULD SEE (3): THE PRESSURE



Pressure (0 to 100 dyn/cm^2) in the same window at $t = 2$ s.

- Above the upper wall the pressure is 49 dyn/cm^2, the p_e the top face imposes; below the lower wall it is about 0, the bottom face's traction. The walls carry the difference.
- In the channel the pressure is 87 upstream of the segment, 62 at its middle, -5 just beyond it and -10 at $x = 20$, then back to about 0 at the free outflow: at this instant almost the whole pressure drop of the channel sits across the segment, because the segment's downstream half is bent in and the released slug is accelerating.
- The segment moves toward the lower pressure: out where the channel's 87 exceeds p_e, in where the channel's -5 is below it. The speckle along the segment is the pressure jump across a thin wall, which the IB kernel smears over a few cells (chapter 19: `use_jump_conditions` is the sharper alternative).

READING ANY OTHER EXAMPLE WITH THE SAME MAP

1. Find `ModelData` (or the functions above `main`): what the body is made of and what moves it. Every example's physics is there, in a few dozen lines.
2. Find every `register`: `grep -n register example.cpp`. Each call is one of chapter 19's hooks (stress, body force, surface force, mapping, initial velocity) on one part, or a fluid-side hook (initial conditions, boundary conditions, body force, writers).
3. Find every `getComponentDatabase("...")`: the list of input blocks the program reads, and so the list of blocks you can change. A block in the input that no call names is dead (slide 28).
4. Find `build_` or `read(`: how the mesh comes to be, and which input keys size it.
5. Find the time loop and what is in its output block beyond the standard four; and what `output_data` (or `postprocess_data`) writes.
6. Date the file by its headers and start-up (slide 45), and update it before you build.

> Chapter 25 applies this to IBAMR's other IBFE examples; chapter 15 did it for the IB ones. The course's own files (chapters 17–22) follow the same map, with `main.C` for `example.cpp`.

WHAT CAN GO WRONG

symptom	cause and fix
`'IBTKInit' was not declared`, or `SAMRAI_MPI::setCommunicator` errors	the 2020 start-up against today's headers, or the reverse; use slide 15's current block (chapter 17)
`'NULL' was not declared` / `AutoPtr` errors	`nullptr`, `std::unique_ptr` (slide 45)
an input block is ignored	its name differs from the `getComponentDatabase` string, or its keys from what the class reads (the sponge layer); check `// input used` in the log
"Unsupported solver type"	`Main.solver_type` is not `"STAGGERED"` or `"COLLOCATED"`
a crash at the very end of a run	an object created outside the inner braces, destroyed after the libraries shut down
the second part has no stiffness	no stress registered on it: in `ex7` that is intended for the lower wall
nothing in `output_data`'s folder	`data_dump_interval = 0` (the default), or the folder name empty
"Simulation time" never advances	the step is tiny: `dt_max`, or a CFL limit from a velocity that blew up
`error_on_dt_change` abort after a restart	the restarted run's `DT` differs from the original's
`mpirun` refuses to start as root	Open MPI wants `--allow-run-as-root` (or the two `OMPI_ALLOW_RUN_AS_ROOT*` variables) in a container or WSL root shell

WHAT THE 2020 FILE HAD THAT IS NOW PART X

2020 (ex7 as the lecture showed it)	0.19.0
`#include <IBAMR_config.h>`, `<IBTK_config.h>`	gone; `<ibtk/IBTKInit.h>`, `<ibtk/IBTK_MPI.h>` added
`LibMeshInit init(argc, argv);`	`IBTKInit ibtk_init(argc, argv, MPI_COMM_WORLD); const LibMeshInit& init = ibtk_init.getLibMeshInit();`
`SAMRAI_MPI::setCommunicator(...);`	
`SAMRAIManager::startup(); ...shutdown()`	
`static_cast<RobinBcCoefStrategy <NDIM>*>(NULL)`	`nullptr`
`std::unique_ptr<ExodusII_IO> io(uses_exodus ? new ExodusII_IO(mesh) : NULL)`	`uses_exodus ? std::make_unique<ExodusII_IO>(mesh) : nullptr`
`MathUtilities<double>::equalEps(loop_time, loop_time_end)`	`IBTK::rel_equal_eps(loop_time, loop_time_end)`
`SAMRAI_MPI::getRank()` in `output_data`	`IBTK_MPI::getRank()`
the lecture's C++ primer from tutorialspoint and the stack from IBAMR's README	slides 4–7, in this series' words
"not totally sure what sponge layer does"	slide 28

EXERCISES

1. Rename the sponge layer's keys to `forcing_enabled_0`, `forcing_enabled_1` and run `caseC` to 0.5 s with and without: compare v in the first cell at the inflow (a VisIt lineout, chapter 29).
2. Add a chapter 22 force integral to `output_data` for the upper wall's tether, set `data_dump_interval`, and plot the force the fluid puts on the rigid parts of the wall against time.
3. Give the lower wall the same stress function as the upper one with its own elastic segment, and watch the channel collapse from both sides.
4. Run `caseS2` to 5 s (a few hours): does the segment begin to oscillate?
5. Replace the walls by a mesh read from a file (chapter 18) with the same geometry, and check that the run is the same.
6. Take the course's `main.C` of chapter 17 and mark its seventeen blocks with the map of slides 10–11; which are missing, and which are extra?

RULES OF THUMB

- Every IBAMR main file is the same seventeen blocks in the same order; learn the map once.
- The physics is in `ModelData` and the `register` calls; everything else is plumbing you copy.
- An input block is read only if a `getComponentDatabase` names it, and a key only if the class spells it the same way; the log's `// input used` tells you.
- Objects live inside the inner braces; `IBTKInit` outside.
- `output_data` is yours: a measurement or a state dump, with the signature kept.
- Date a file by its headers; update the start-up, `nullptr` and `unique_ptr` before you build.

SUMMARY

1. `ex7` is a collapsible channel: two thin tethered walls built in the program, an elastic St. Venant–Kirchhoff segment in the upper one, a ramped parabolic inflow, a free outflow, and an external pressure applied on the top face of the box; three cases at $\mathrm{Re} = 500$, 200 and 77.
2. The file is seventeen blocks: headers, model callbacks, start-up (`IBTKInit`), `AppInitializer` and the `Main` keys with their aliases, meshes, solvers, the six grid objects, the IBFE registrations per part, initial and boundary conditions from muParser, a body force, writers, initialization, the time loop with its four output `if`s, and `output_data`.
3. The sponge layer damps a velocity component near a face; in `ex7` its keys are misspelt and it is off.
4. Run to 2 s on four cores in an hour: the ramp raises the upstream pressure above p_e, the segment bulges outward by 0.6 cm, then swings back and pumps a slug of fluid down the channel.
5. The 2020 version differs in the start-up, `NULL`, `unique_ptr` and the time test; the map, and the recipe for reading any other example, are the same.

Chapter 24: IBFE with advection–diffusion, a concentration carried by the flow past a moving body.

RECORDINGS (LISTED, NOT INCORPORATED)

The 2020 Zoom recording of *Lecture 8-IBFE-Going through a main file* is listed in the front matter's video list (chapter 0.1). The lecture's nine code screenshots are transcribed on slides 12–34 from the 0.19.0 file, with the 2020 differences on slide 45.

REFERENCES AND CREDITS

- IBAMR 0.19.0: `examples/IBFE/explicit/ex7/example.cpp` and its three inputs; `ibtk/src/utilities/AppInitializer.cpp`, `IBTKInit.cpp`; `src/navier_stokes/SpongeLayerForceFunction.cpp`; `include/ibamr/app_namespaces.h`. https://github.com/IBAMR/IBAMR; documentation https://ibamr.github.io/docs.
- The course material: *Lecture 8-IBFE-Going through a main file* (2020; its code screenshots and notes).
- libMesh: `MeshTools::Generation::build_square`, https://libmesh.github.io. SAMRAI, PETSc, hypre, muParser, HDF5: the libraries of slides 4–5.
- X. Y. Luo and T. J. Pedley, *A numerical simulation of unsteady flow in a two-dimensional collapsible channel*, J. Fluid Mech. 314 (1996) 191–225: the geometry and the external-pressure set-up that `ex7` follows.
- The run: `/root/work/0.19.0/ibfe_ex7` (IBAMR 0.19.0, four cores); figures with VisIt 3.5, `wsl/final/visit_ch23.sh`.